Unit 4 – Partially Observable Markov Decision Processes and the Bayes' Filter

4SC000 Optimal Control and Reinforcement Learning

Duarte Antunes

Slides · Exercises · Jupyter notebooks

Problem formulation

In several control problems considered so far, we implicitly assumed that the full state x_k is available for feedback. In many practical situations this is not the case: only an *output*, a subset or a noisy function of the state, can actually be measured. For instance, when controlling the position of a mass on a surface using only position sensors, the state – which also includes velocity – is not fully known. More generally, the available output might carry only partial information about the state. This unit studies *Partially Observable Markov Decision Processes* (POMDPs), the framework for optimal control under this kind of limited information, and the *Bayes' filter*, the tool that turns a stream of partial measurements into a running estimate of the state. Most of the unit is devoted to the Bayes' filter itself, since it is useful well beyond the control problems that motivate it here.

A POMDP has the same dynamic model and cost as the discrete decision problems of Unit 2,

$$x_{k+1} = f_k(x_k, u_k, w_k), \qquad \text{cost} = \sum_{k=0}^{h-1} g_k(x_k, u_k, w_k) + g_h(x_h),$$

but now the decision-maker does not observe x_k directly. Instead, at every stage k it receives an *output*

$$y_k = h_k(x_k, n_k),$$

where n_k is measurement noise, and the state, input and disturbances live in the same sets as in Unit 2, while the output y_k and the noise n_k live in finite sets $Y_k := \{1, \dots, q_{k,i}\}$, $N_k := \{1, \dots, \nu_{k,i}\}$ (possibly depending on the current state $x_k = i$). All the information available to the decision-maker at stage k is collected in the *information set*

$$I_0 = (y_0), \qquad I_k = (y_0, y_1, \dots, y_k, u_0, u_1, \dots, u_{k-1}), \quad k \geq 1,$$

and decisions must be functions of this information set, $u_k = \mu_k(I_k)$, rather than of the state itself. The problem is to find a policy $\pi = \{\mu_0, \dots, \mu_{h-1}\}$ minimizing

$$J_\pi = \mathbb{E}\Big[\sum_{k=0}^{h-1} g_k(x_k, \mu_k(I_k), w_k) + g_h(x_h)\Big].$$

Quick exercise: partial information in inventory control and robot path planning

Justify a model with partial state information for the inventory control problem and the robot path planning problem considered in Unit 2, and write such problems in the framework just described.

Solution

Inventory control. A shop rarely knows its stock level x_k exactly and in real time. Physical counts are typically done periodically rather than continuously, and even then they are not perfectly reliable: items can be misplaced, damaged, or stolen without this being recorded, and barcode or RFID scanners occasionally fail to register an item during a count. So what is actually available at each stage is a noisy count of the stock, not the true value x_k.
The dynamic model and cost are unchanged from Units 1-2, with the demand d_k playing the role of the disturbance w_k,

$$x_{k+1} = f_k(x_k, u_k, w_k) = \max\{x_k + u_k - w_k,\, 0\}, \qquad u_k \in \{0, 1, \dots, N - x_k\},$$

$$g_k(x_k, u_k, w_k) = (c_1(x_k) + c\,u_k + c_{tr}\|u_k\|_0) - p\min\{w_k, x_k + u_k\}.$$

For the output, model the periodic count as being off by one item with some small probability q, independently of everything else (a single missed or misplaced item being the typical failure mode, rather than large miscounts):

$$y_k = h_k(x_k, n_k) = \max\{x_k - n_k,\, 0\}, \qquad n_k \in \{0, 1\}, \qquad \text{Prob}[n_k = 1] = q, \quad \text{Prob}[n_k = 0] = 1 - q,$$

so that $y_k \in Y_k = \{0, \dots, N\}$, exactly as required by the general framework. The information set is then $I_k = (y_0, \dots, y_k, u_0, \dots, u_{k-1})$, and the shop must decide how much to order, $u_k = \mu_k(I_k)$, based on this history of noisy counts and past orders rather than on the true (unknown) stock level x_k.
Robot path planning. A robot moving through the grid of Unit 2 likewise has no direct way of reading off its exact row x_k: there is typically no GPS indoors, and dead-reckoning from wheel encoders drifts over time. A cheap sensor, however – e.g. an infrared proximity sensor – can reliably detect whether the robot is currently forced against a wall or an obstacle, without revealing which row it is actually in. This is exactly the same kind of binary, "contact-only" output already used for Sensor II in this unit's grid-world example.
The dynamic model and cost are again unchanged from Unit 2's (deterministic- or stochastic-disturbance) robot-grid problem,

$$x_{k+1} = f_k(x_k, u_k, w_k), \qquad g_k(x_k, u_k, w_k),$$

with $x_k = i \in \{1, \dots, n\}$ the row and $u_k \in \{\text{up}, \text{straight}, \text{down}\}$ the decision (a singleton at wall or obstacle rows). Writing $\text{touch}(i) = 1$ if row i is a wall row ($i \in \{1, n\}$) or an obstacle row, and $\text{touch}(i) = 0$ otherwise, the output is a possibly-flipped version of this contact bit,

$$y_k = h_k(x_k, n_k) = \text{touch}(x_k) \oplus n_k, \qquad n_k \in \{0, 1\}, \qquad \text{Prob}[n_k = 1] = \varepsilon, \quad \text{Prob}[n_k = 0] = 1 - \varepsilon,$$

where $\oplus$ denotes addition modulo 2 and ε is a small sensor-error probability, so that $y_k \in Y_k = \{0, 1\}$. The information set is $I_k = (y_0, \dots, y_k, u_0, \dots, u_{k-1})$, and the robot chooses its next move $u_k = \mu_k(I_k)$ based only on this history of (possibly mistaken) wall/obstacle contact readings and its own past decisions, not on its actual row x_k.

Two approaches, and the role of the Bayes' filter

A first, direct approach is to reformulate the partial-information problem as an ordinary full-information problem by treating the *information set itself* as the state: since I_k evolves in a known way as y_{k+1} and u_k are appended to it, dynamic programming applies directly, with I_k playing the role of x_k. The trouble is that the size of the information set grows very quickly with k – already after just two stages, the space of possible

information sets is large, as illustrated in Fig. 1 – so the number of "states" of this reformulated problem grows exponentially with time, making the direct approach impractical for essentially any application of interest.

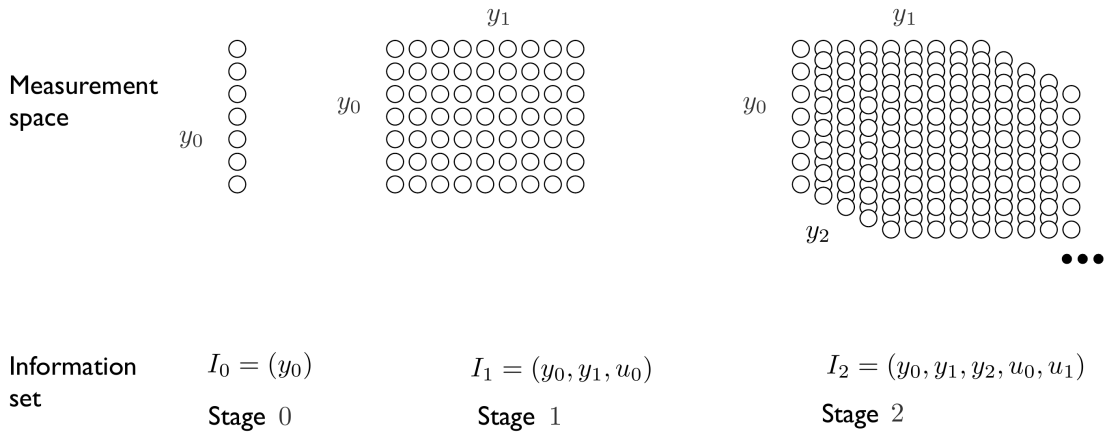


Fig. 1: The measurement/information space grows quickly with the stage: at stage 0 there are as many possible information sets as values of y_0; at stage 1, as many as pairs (y_0, y_1); and so on.

A second, much more economical approach exploits a key fact: it can be shown that the conditional probability distribution of the current state given all information gathered so far, $P_{x_k|I_k}$, is a *sufficient statistic* for optimal decision-making – that is, an optimal policy can always be written as $u_k = \mu_k(P_{x_k|I_k})$, a function of this distribution rather than of the raw information set I_k itself (Fig. 2). Since $P_{x_k|I_k}$ lives in a fixed-dimensional space (the simplex of probability distributions over the states), regardless of k, this sidesteps the exponential blow-up of the first approach entirely.

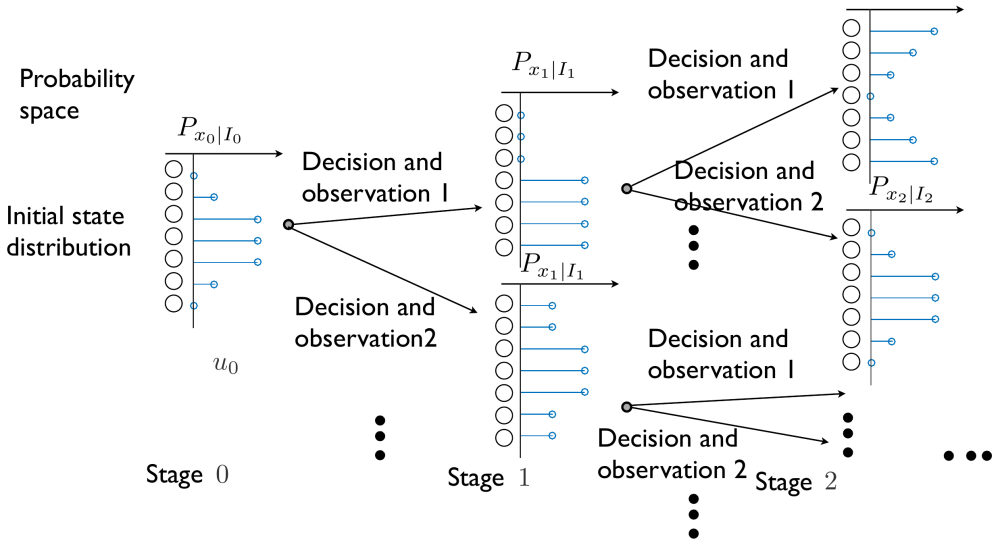


Fig. 2: As decisions and observations accumulate, the belief $P_{x_k|I_k}$ always lives in the same fixed-dimensional probability space, however the raw information set I_k itself keeps growing.

This has an appealing structural consequence for the optimal decision-maker, proved at the end of this section: it can always be decomposed into a *state estimator*, which computes $P_{x_k|I_k}$ from the measurements y_k and past decisions u_{k-1}, and an *actuator*, which maps this belief to a decision $u_k = \mu_k(P_{x_k|I_k})$ (Fig. 3). This is exactly the classical *separation* between estimation and control that recurs throughout control theory.

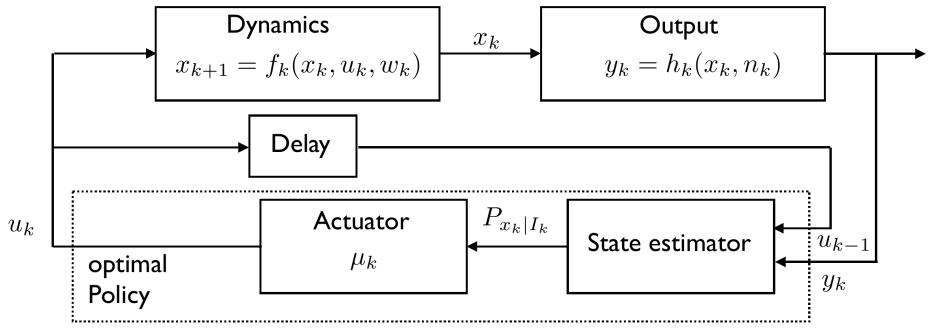


Fig. 3: The optimal POMDP policy decomposes into a state estimator, producing the belief $P_{x_k|I_k}$, and an actuator (the optimal policy μ_k) mapping that belief to a decision u_k.

The first approach is therefore typically impractical, while the second is the one used in practice; a crucial ingredient in computing $P_{x_k|I_k}$ is Bayes' rule, and the resulting state estimator is precisely the Bayes filter, which is the focus of most of this unit. How to solve POMDPs – i.e., how to obtain the actuator μ_k – is addressed only briefly, toward the end.

It will be convenient throughout to work with the transition probabilities already introduced in Unit 2,

$$[P_k(j)]_{ri} = \text{Prob}[x_{k+1} = r \mid x_k = i, u_k = j],$$

which collapses to a single matrix P if there is no input and the model is time-invariant. Likewise, the output model $y_k = h_k(x_k, n_k)$ can be represented by matrices

$$[R_k]_{\ell i} = \text{Prob}[y_k = \ell \mid x_k = i],$$

which collapses to a single matrix R if the output is time-invariant. Assuming, for simplicity, that the input, disturbance, output and noise spaces do not depend on the state (though they may still depend on time), $x_k \in \{1, \dots, \bar{n}_k\}$, $u_k \in \{1, \dots, m_k\}$, $w_k \in \{1, \dots, \omega_k\}$, $y_k \in \{1, \dots, \bar{q}_k\}$, $n_k \in \{1, \dots, \nu_k\}$, these matrices have the following dimensions, for each $j \in \{1, \dots, m_k\}$ and each $k \in \{0, \dots, h-1\}$:

	dimensions	Components
state transition matrices $P_k(j)$	$\bar{n}_{k+1} \times \bar{n}_k$	$[P_k(j)]_{ri} = \text{Prob}[x_{k+1} = r \mid x_k = i, u_k = j]$
output matrices R_k	$\bar{q}_k \times \bar{n}_k$	$[R_k]_{\ell i} = \text{Prob}[y_k = \ell \mid x_k = i]$

i Proof: the belief is a sufficient statistic, and the optimal decision-maker decomposes into an estimator and an actuator

We prove the claim made above: the optimal cost-to-go, and an optimal action, at every stage k depend on the information set I_k only through the belief $p_{k,i} := \text{Prob}[x_k = i \mid I_k]$, $i \in \{1, \dots, \bar{n}_k\}$. In particular, the optimal policy can always be written as a function of p_k alone, $u_k = \tilde{\mu}_k(p_k)$ (the *actuator*), and p_k can itself be updated recursively from p_{k-1}, u_{k-1} and y_k alone, without needing the rest of I_k (the *state estimator*).
Consider the full-information reformulation of the POMDP with I_k as the state, introduced above. Applying the DP algorithm to this reformulation, backward from $k = h$,

$$J_h(I_h) = \mathbb{E}[g_h(x_h) \mid I_h],$$

$$J_k(I_k) = \min_{u_k} \mathbb{E}[g_k(x_k, u_k, w_k) + J_{k+1}(I_{k+1}) \mid I_k, u_k], \quad k \in \{h-1, \dots, 0\}.$$

We show, by backward induction on k, that $J_k(I_k)$ depends on I_k only through p_k; whenever this holds, write $J_k(I_k) = \tilde{J}_k(p_k)$.
Base case. At $k = h$,

$$J_h(I_h) = \mathbb{E}[g_h(x_h) \mid I_h] = \sum_{i=1}^{\bar{n}_h} g_h(i)\, p_{h,i} =: \tilde{J}_h(p_h),$$

manifestly a function of p_h alone.
Inductive step. Assume $J_{k+1}(I_{k+1}) = \tilde{J}_{k+1}(p_{k+1})$. Splitting the expectation defining $J_k(I_k)$ into its two terms: first,

$$\mathbb{E}[g_k(x_k, u_k, w_k) \mid I_k, u_k] = \sum_{i=1}^{\bar{n}_k} p_{k,i}\, \bar{g}_k(i, u_k), \qquad \bar{g}_k(i, u_k) := \mathbb{E}_{w_k}[g_k(i, u_k, w_k)],$$

already a function of p_k and u_k alone. Second, condition on the next output $y_{k+1} = \ell$: by the Markov property of the state transitions and of the output map,

$$\text{Prob}[x_{k+1} = r,\, y_{k+1} = \ell \mid I_k, u_k] = [R_k]_{\ell r} \sum_{i=1}^{\bar{n}_k} [P_k(u_k)]_{ri}\, p_{k,i},$$

which depends on I_k only through p_k. Summing over r gives $\text{Prob}[y_{k+1} = \ell \mid I_k, u_k]$, and dividing gives the updated belief after observing ℓ,

$$p_{k+1,r}(\ell) = \frac{[R_k]_{\ell r} \sum_{i=1}^{\bar{n}_k} [P_k(u_k)]_{ri}\, p_{k,i}}{\text{Prob}[y_{k+1} = \ell \mid I_k, u_k]},$$

both again functions of (p_k, u_k, ℓ) alone – and no other part of I_k. Hence

$$\mathbb{E}[J_{k+1}(I_{k+1}) \mid I_k, u_k] = \sum_{\ell} \text{Prob}[y_{k+1} = \ell \mid p_k, u_k]\, \tilde{J}_{k+1}(p_{k+1}(\ell))$$

is also a function of p_k and u_k alone. Adding the two terms and minimizing over u_k,

$$J_k(I_k) = \min_{u_k} \Big[\sum_i p_{k,i}\, \bar{g}_k(i, u_k) + \sum_{\ell} \text{Prob}[y_{k+1} = \ell \mid p_k, u_k]\, \tilde{J}_{k+1}(p_{k+1}(\ell)) \Big] =: \tilde{J}_k(p_k),$$

depends on I_k only through p_k, completing the induction.
In particular, a minimizer u_k in the last display is itself a function of p_k alone, so the optimal policy is $u_k = \tilde{\mu}_k(p_k)$: this is the *actuator*. Moreover, the update formula for $p_{k+1}(\ell)$ derived above shows that the belief can be maintained online using only p_k, u_k, and the realized output y_{k+1} – never needing to store or revisit the rest of I_k: this recursive update is exactly the Bayes filter, worked out concretely in the next section, and is the *state estimator*. Together, these two facts establish the claimed decomposition.

The Bayes' filter

Bayes' rule

The Bayes filter is built on the ordinary Bayes' rule from probability theory. For random variables $x \in \{1, \dots, n\}$, $y \in \{1, \dots, m\}$,

$$\text{Prob}[x = i \mid y = j] = \frac{\text{Prob}[y = j \mid x = i]\, \text{Prob}[x = i]}{\text{Prob}[y = j]} = \frac{\text{Prob}[y = j \mid x = i]\, \text{Prob}[x = i]}{\sum_{\ell=1}^{n} \text{Prob}[y = j \mid x = \ell]\, \text{Prob}[x = \ell]}.$$

Thinking of y as a sensor measurement and x as the state (Fig. 4), Bayes' rule lets us infer something

about the state *a posteriori* of a measurement, $\text{Prob}[x = i \mid y = j]$, starting from *a priori* information $\text{Prob}[y = j \mid x = i]$ (how the sensor behaves in each state) and $\text{Prob}[x = i]$ (how likely each state was to begin with).

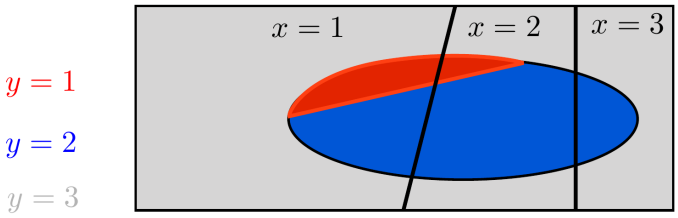


Fig. 4: A joint probability space for a discrete state $x \in \{1, 2, 3\}$ and a discrete measurement $y \in \{1, 2, 3\}$: the colored region shows how probability mass is shared between the possible values of y within each value of x.

Quick exercise: reading conditional probabilities off the picture

In Fig. 4 the whole rectangle is the sample space, split by the two vertical lines into the strips $x = 1$, $x = 2$ and $x = 3$. Inside the ellipse, the red part is $y = 1$ and the blue part is $y = 2$; everything outside the ellipse (the grey background) is $y = 3$.
Writing $|R|$ for the area of a region R, and $\{x = 3\}$ for the strip $x = 3$, which of the following gives $\text{Prob}[y = j \mid x = 3]$ for $j \in \{1, 2, 3\}$? Exactly one is correct.

(a)

$$\begin{aligned}
\text{Prob}[y = 1 \mid x = 3] &= 0, \\
\text{Prob}[y = 2 \mid x = 3] &= \frac{|\text{blue} \cap \{x = 3\}|}{|\text{whole rectangle}|}, \\
\text{Prob}[y = 3 \mid x = 3] &= \frac{|\text{grey} \cap \{x = 3\}|}{|\text{whole rectangle}|}.
\end{aligned}$$

(b)

$$\begin{aligned}
\text{Prob}[y = 1 \mid x = 3] &= 0, \\
\text{Prob}[y = 2 \mid x = 3] &= \frac{|\text{blue} \cap \{x = 3\}|}{|\{x = 3\}|}, \\
\text{Prob}[y = 3 \mid x = 3] &= \frac{|\text{grey} \cap \{x = 3\}|}{|\{x = 3\}|}.
\end{aligned}$$

(c)

$$\begin{aligned}
\text{Prob}[y = 1 \mid x = 3] &= 0, \\
\text{Prob}[y = 2 \mid x = 3] &= \frac{|\text{blue} \cap \{x = 3\}|}{|\text{blue}|}, \\
\text{Prob}[y = 3 \mid x = 3] &= \frac{|\text{grey} \cap \{x = 3\}|}{|\text{grey}|}.
\end{aligned}$$

Solution

(b) is correct.
All three options agree that $\text{Prob}[y = 1 \mid x = 3] = 0$, and that much is simply read off the figure: the red region lies entirely to the left of the second vertical line, so it has no overlap at all with the strip

$x = 3$. A measurement $y = 1$ is impossible when the state is 3.
The three options differ only in what the remaining areas are divided by, and that is the whole point of the exercise. Conditioning on $x = 3$ means *restricting attention to the strip $x = 3$ and treating it as the new sample space*, so the areas must be normalized by the area of that strip. Within the strip the ellipse contributes only blue, and the rest is grey, so the two probabilities add up to 1 as they must.
Option **(a)** divides by the area of the whole rectangle instead. Those are the *joint* probabilities $\text{Prob}[y = j \text{ and } x = 3]$, not the conditional ones; they add up to $\text{Prob}[x = 3]$, which is less than 1.
Option **(c)** normalizes each measurement value by its own total area, which conditions the wrong way round: it computes $\text{Prob}[x = 3 \mid y = j]$, the probability of the state given the measurement, rather than the probability of the measurement given the state. Keeping these two apart is exactly what Bayes' rule is for – the rule above is precisely the recipe for turning one into the other.

Quick exercise: now the other way round

Still reading off Fig. 4, and with the same notation $|R|$ for the area of a region R, compute $\text{Prob}[x = 2 \mid y = 1]$ in terms of areas of regions in the figure.

Solution

$$\text{Prob}[x = 2 \mid y = 1] = \frac{|\text{red} \cap \{x = 2\}|}{|\text{red}|}.$$

Conditioning on $y = 1$ means restricting attention to the red region and treating *it* as the new sample space – exactly as conditioning on $x = 3$ in the previous exercise meant restricting attention to the strip $x = 3$. The event $x = 2$ then corresponds to the part of the red region lying to the right of the first vertical line, and its probability is that part's share of the red region as a whole.
Note that this is the quantity option (c) of the previous exercise was really computing. It is the *a posteriori* probability of the state given the measurement – what we actually want when a sensor reading arrives – whereas $\text{Prob}[y = j \mid x = i]$ is the sensor model we are given. Bayes' rule is what connects the two, and in this picture it is the almost trivial observation that both are ratios of the same overlapping area, differing only in what they are normalized by.

A one-shot application of Bayes' rule: should she buy the alarm?

Bayes' rule is useful even outside of a filtering/tracking context, whenever a single a-posteriori probability needs to be computed from a single measurement. Here is a simple illustration.
An international student is deciding whether it is worthwhile to install a hobby alarm (an ultrasound sensor near the door) to detect burglars while she is away for the Christmas break. The alarm is faulty and characterized by

	Burglar breaks in B	Burglar does not break in $\neg B$
Alarm does not go off $\neg A$	$\text{Prob}[\neg A \mid B] = 0.01$	$\text{Prob}[\neg A \mid \neg B] = 0.9$
Alarm goes off A	$\text{Prob}[A \mid B] = 0.99$	$\text{Prob}[A \mid \neg B] = 0.1$

and historical data shows that 2 out of every 100 rooms are burgled each Christmas, i.e. $\text{Prob}[B] = 0.02$, $\text{Prob}[\neg B] = 0.98$. Her belongings are worth 2000 euros, and calling a security agency to check the room whenever the alarm goes off costs 300 euros; she will only call if $\text{Prob}[B \mid A] \cdot 2000 > 300$, i.e. if $\text{Prob}[B \mid A] > 0.15$. Should she buy the alarm?
By Bayes' rule,

$$\text{Prob}[B \mid A] = \frac{\text{Prob}[A \mid B]\,\text{Prob}[B]}{\text{Prob}[A \mid B]\,\text{Prob}[B] + \text{Prob}[A \mid \neg B]\,\text{Prob}[\neg B]} = \frac{0.99 \times 0.02}{0.99 \times 0.02 + 0.1 \times 0.98} = \frac{0.0198}{0.1178} \approx 0.168,$$

which exceeds 0.15 – so yes, she should buy the alarm (and call the agency whenever it goes off).

Example: tracking a robot behind a door

A robot can be at one of 8 positions on a circular floor with a door (Fig. 5). It moves around the circle with some slip, according to

$$\begin{aligned}
\text{Prob}[p_{k+1} = i \mid p_k = i] &= 0.1, \\
\text{Prob}[p_{k+1} = i \oplus 2 \mid p_k = i] &= 0.2, \\
\text{Prob}[p_{k+1} = i \oplus 1 \mid p_k = i] &= 0.7,
\end{aligned}$$

where $i \oplus r := ((i - 1 + r) \bmod 8) + 1$ wraps around the 8 positions.

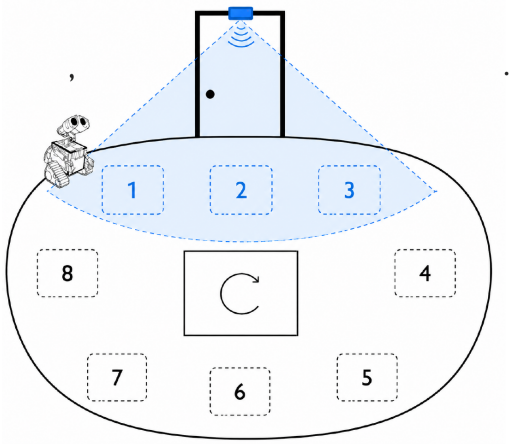


Fig. 5: A robot moving in a circular fashion among 8 positions on a floor with a door; a sensor may detect the door when the robot is close to it.

If the robot is close to the door, a sensor might detect it ($y_k = 1$) with

$$\text{Prob}[y_k = 1 \mid p_k = 2] = 1, \qquad \text{Prob}[y_k = 1 \mid p_k = 1] = \text{Prob}[y_k = 1 \mid p_k = 3] = 0.4,$$

and if $p_k \notin \{1, 2, 3\}$ then $y_k = 0$ with probability one. Suppose the robot starts at $p_0 = 1$; we would like to compute $\text{Prob}[p_2 \mid p_0 = 1]$ and, once a measurement is available, $\text{Prob}[p_2 \mid p_0 = 1, y_2 = 1]$.

The first quantity can be obtained by simply following the branching probabilities two steps forward, as in Fig. 6.

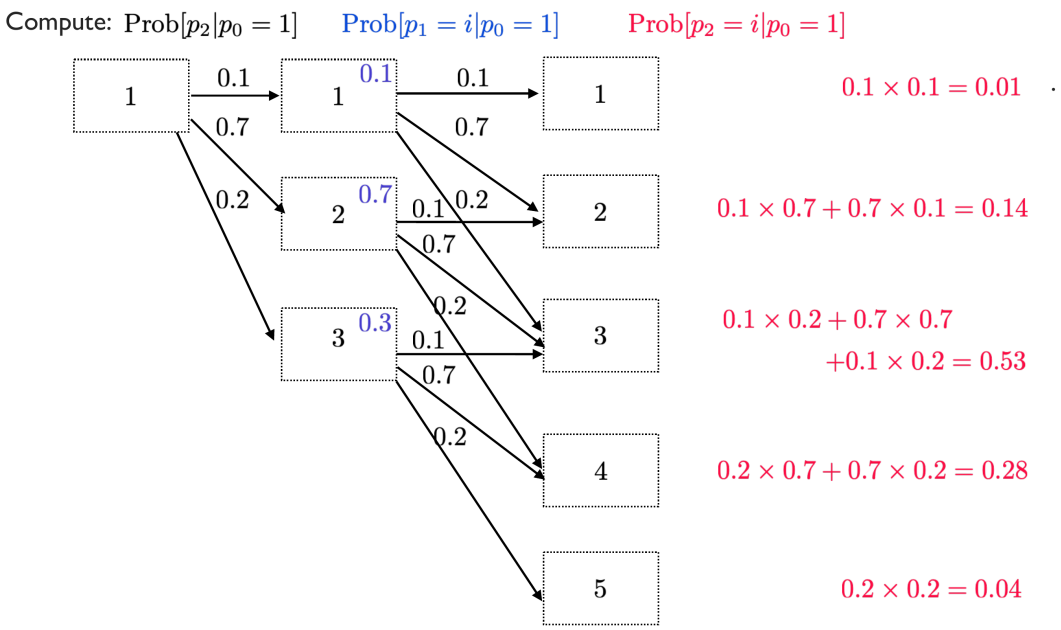


Fig. 6: Propagating Prob[$p_0 = 1$] two steps forward through the transition probabilities $0.1, 0.2, 0.7$ to obtain Prob[$p_2 = i \mid p_0 = 1$] for every i.

Equivalently, and more systematically, define the transition matrix $P_{ij} = \text{Prob}[x_{k+1} = i \mid x_k = j]$ and the probability vector p_k with entries Prob[$x_k = i \mid x_0 = 1$]:

$$P = \begin{bmatrix} 0.1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.7 & 0.1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.2 & 0.7 & 0.1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.2 & 0.7 & 0.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.2 & 0.7 & 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.2 & 0.7 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.2 & 0.7 & 0.1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.2 & 0.7 & 0.1 \end{bmatrix},$$

$$p_0 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \qquad p_1 = \begin{bmatrix} 0.1 \\ 0.7 \\ 0.2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad p_2 = \begin{bmatrix} 0.01 \\ 0.14 \\ 0.53 \\ 0.28 \\ 0.04 \\ 0 \\ 0 \\ 0 \end{bmatrix},$$

obtained simply by iterating $p_{k+1} = Pp_k$.

To fold in the measurement $y_2 = 1$, apply Bayes' rule (with normalizing factor $\alpha = 1/\text{Prob}[y_2 = 1 \mid p_0 = 1]$):

$$\text{Prob}[p_2 = i \mid p_0 = 1, y_2 = 1] = \frac{1}{\alpha} \text{Prob}[y_2 = 1 \mid p_0 = 1, p_2 = i] \, \text{Prob}[p_2 = i \mid p_0 = 1].$$

Writing the sensor model as a diagonal matrix $R = \text{diag}(0.4,\ 1,\ 0.4,\ 0,\ 0,\ 0,\ 0,\ 0)$ (the probability of $y_2 = 1$ at each possible position), this becomes a simple matrix-vector product followed by a normalization:

$$Rp_2 = \begin{bmatrix} 0.004 \\ 0.14 \\ 0.212 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad \alpha = 0.004 + 0.14 + 0.212 = 0.356,$$

$$\text{Prob}[p_2 \mid p_0 = 1, y_2 = 1] = \begin{bmatrix} 0.0112 \\ 0.3933 \\ 0.5955 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

In general, multiplying by the diagonal matrix $D(y_k) = \text{diag}([R]_{y_k,1}, \ldots, [R]_{y_k,\bar{n}})$ – apart from the normalizing factor – is exactly how a measurement updates the belief, and this is the essence of the *correction* step of the Bayes filter, formalized next.

Quick exercise: a different starting position and measurement

Repeat the computation above for a robot that starts at $p_0 = 2$ instead of $p_0 = 1$, and suppose that at stage 2 the door is *not* detected, $y_2 = 0$, instead of detected. Compute $\text{Prob}[p_2 \mid p_0 = 2, y_2 = 0]$.

Solution

Prediction. Iterating $p_{k+1} = Pp_k$ from $p_0 = 2$ (i.e., $p_0 = [0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^\top$),

$$p_1 = \begin{bmatrix} 0 \\ 0.1 \\ 0.7 \\ 0.2 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad p_2 = \begin{bmatrix} 0 \\ 0.01 \\ 0.14 \\ 0.53 \\ 0.28 \\ 0.04 \\ 0 \\ 0 \end{bmatrix}.$$

This is exactly the p_2 found earlier for $p_0 = 1$, shifted by one position around the circle – as expected, since the dynamics (unlike the sensor model) are the same at every position.

Correction. Since $y_2 = 0$ instead of $y_2 = 1$, the relevant sensor row is now $\text{Prob}[y_2 = 0 \mid p_2 = i] = 1 - \text{Prob}[y_2 = 1 \mid p_2 = i]$, i.e. the diagonal matrix $D(0) = \text{diag}(0.6,\ 0,\ 0.6,\ 1,\ 1,\ 1,\ 1,\ 1)$. Then

$$D(0)\, p_2 = \begin{bmatrix} 0 \\ 0 \\ 0.084 \\ 0.53 \\ 0.28 \\ 0.04 \\ 0 \\ 0 \end{bmatrix}, \qquad \alpha = 0.084 + 0.53 + 0.28 + 0.04 = 0.934,$$

$$\text{Prob}[p_2 \mid p_0 = 2, y_2 = 0] = \begin{bmatrix} 0 \\ 0 \\ 0.0899 \\ 0.5675 \\ 0.2998 \\ 0.0428 \\ 0 \\ 0 \end{bmatrix}.$$

Two things are worth noting. First, position 2 (where the door sensor would fire with certainty) is ruled out entirely once $y_2 = 0$ is observed, and position 1 also drops out since it started with negligible mass. Second, compared to the *prediction* p_2 above, the belief shifts further away from the door (positions 1-3) and concentrates more on position 4, since observing "no door" is evidence *against* being close to it.

! Interactive example: run the Bayes filter yourself

The interactive panel below lets you run the Bayes filter yourself on the robot-and-door example of Fig. 5 (the very same image is used here). Choose an initial belief p_0 over the 8 positions – any nonnegative numbers, automatically normalized to sum to one – then build a sequence of measurements ($y = 1$: door detected, $y = 0$: door not detected), one per stage. "Step" advances the filter by one half-step at a time (first the *prediction*, then the *correction* using the next measurement in the sequence); "Play" runs through the whole sequence automatically. The number shown at each position, and the intensity of its highlight, is the current belief $\text{Prob}[x_k = i \mid I_k]$ at that position.

The Bayes' filter algorithm

Write the belief $P_{x_k|I_k}$ as the vector $p_k = \begin{bmatrix} p_{k,1} & \dots & p_{k,\bar{n}} \end{bmatrix}^\top$, $p_{k,i} = \text{Prob}[x_k = i \mid I_k]$. When there is no input and the model is time-invariant (Fig. 7), the Bayes filter alternates a *prediction* step, which propagates the belief through the dynamics, and a *correction* step, which folds in the new measurement:

Prediction $\bar{p}_{k+1} = P p_k$

Correction $q_{k+1} = D(y_{k+1})\bar{p}_{k+1} \qquad p_{k+1} = \dfrac{q_{k+1}}{\mathbb{1}^\top q_{k+1}}$

Initial condition $p_0 = \dfrac{q_0}{\mathbb{1}^\top q_0} \qquad q_0 = D(y_0)\tilde{p}_0 \qquad \tilde{p}_{0,i} = \text{Prob}[x_0 = i]$

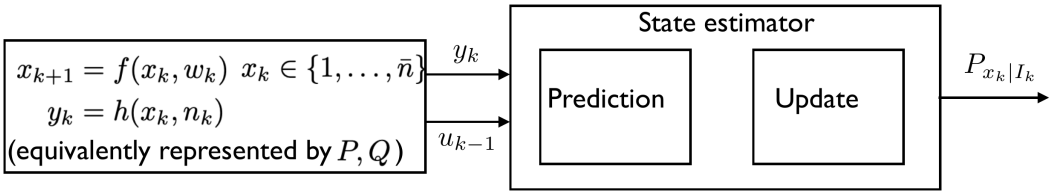


Fig. 7: The Bayes filter (no input, time-invariant case): a state estimator that alternates a prediction step and an update (correction) step to produce the belief $P_{x_k|I_k}$ from the measurements y_k.

In the general case, with an input and possibly time-varying dynamics and output maps (Fig. 8), the prediction step simply uses the transition matrix corresponding to the decision actually taken:

Prediction $\bar{p}_{k+1} = P_k(u_k) p_k$

Correction $q_{k+1} = D(y_{k+1})\bar{p}_{k+1} \qquad p_{k+1} = \dfrac{q_{k+1}}{\mathbb{1}^\top q_{k+1}}$

Initial condition $p_0 = \dfrac{q_0}{\mathbb{1}^\top q_0}$ $\qquad q_0 = D(y_0)\tilde{p}_0$ $\qquad \tilde{p}_{0,i} = \text{Prob}[x_0 = i]$

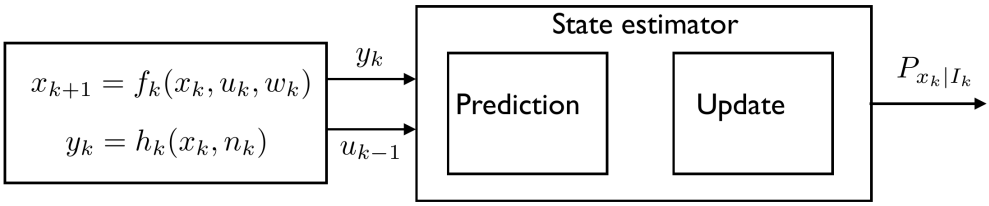


Fig. 8: The Bayes filter, general case: dynamics and output map may depend on time and on the applied input u_k.

i Derivation of the Bayes filter

Both steps of the filter follow directly from the Markov property and Bayes' rule.
Prediction step. Define $\bar{p}_k = \begin{bmatrix}\bar{p}_{k,1} & \dots & \bar{p}_{k,\bar{n}_k}\end{bmatrix}^\top$, $\bar{p}_{k,i} = \text{Prob}[x_k = i \mid I_{k-1}, u_{k-1}]$. Conditioning on x_k,

$$\bar{p}_{k+1,i} = \text{Prob}[x_{k+1} = i \mid I_k, u_k] = \sum_{\tau=1}^{\bar{n}_k} \text{Prob}[x_{k+1} = i \mid x_k = \tau, I_k, u_k]\,\text{Prob}[x_k = \tau \mid I_k, u_k].$$

Since x_k is independent of u_k (the decision at stage k cannot influence the state already reached at stage k), and by the Markov property $\text{Prob}[x_{k+1} = i \mid x_k = \tau, I_k, u_k] = \text{Prob}[x_{k+1} = i \mid x_k = \tau, u_k] = [P_k(u_k)]_{i\tau}$, this becomes

$$\bar{p}_{k+1,i} = \sum_{\tau=1}^{\bar{n}_k} [P_k(u_k)]_{i\tau}\, p_{k,\tau}, \qquad \text{i.e., in matrix form} \qquad \bar{p}_{k+1} = P_k(u_k)p_k.$$

Correction step. Suppose a measurement $y_{k+1} = \theta$ arrives. Then, by Bayes' rule (conditioning on the third variable I_k, u_k) and again the Markov property (y_{k+1} depends on x_{k+1} only),

$$p_{k+1,i} = \text{Prob}[x_{k+1} = i \mid I_k, y_{k+1} = \theta, u_k] = \alpha\,\text{Prob}[y_{k+1} = \theta \mid x_{k+1} = i]\,\text{Prob}[x_{k+1} = i \mid I_k, u_k],$$

with $\alpha = 1/\text{Prob}[y_{k+1} = \theta \mid I_k, u_k]$. In matrix form, $p_{k+1} = \alpha q_{k+1}$ with $q_{k+1} = D(y_{k+1})\bar{p}_{k+1}$; since p_{k+1} must sum to one, $\alpha = 1/\mathbb{1}^\top q_{k+1}$, giving

$$p_{k+1} = \frac{q_{k+1}}{\mathbb{1}^\top q_{k+1}}.$$

Applications of the Bayes filter

Tracking a robot in a grid world

A more involved, but structurally identical, example is tracking the position of a robot moving in a grid-world environment. Every cell of the environment is labeled $i \in \{1, \dots, n\}$, and $x_k = i$ indicates that the robot's position coincides with the centre of cell i (Fig. 9). The robot moves according to a fixed, autonomous policy, $x_{k+1} = f(x_k, w_k)$ (there is no control input in this example, so the derivations above specialize immediately, simply dropping u_k). Without disturbances, the motion follows a fixed *vector field* (Fig. 10).

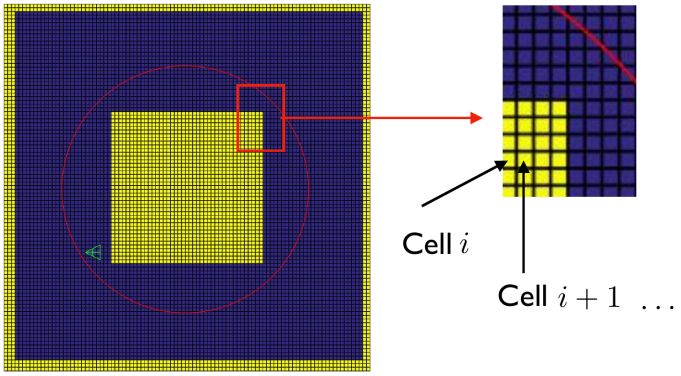


Fig. 9: A grid-world environment: every cell is labeled $i \in \{1, \dots, n\}$, and the robot's state x_k is the cell it currently occupies.

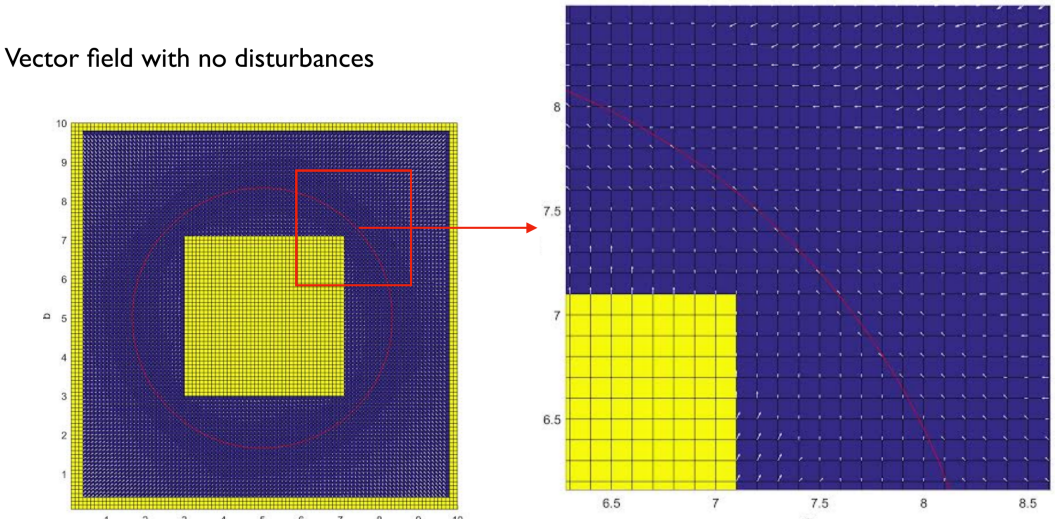


Fig. 10: The deterministic vector field driving the robot's motion (no disturbances), zoomed in on a small region of the grid on the right.

With disturbances, the next state $x_{k+1} = f(x_k, w_k)$ is uncertain and takes values in a neighbourhood of the deterministic next state $f(x_k, 0)$, with probabilities that decay with distance (Fig. 11): the most likely outcome, $p_1 = 0.65$, is the deterministic one, while cells 1, 2, and 3 steps away receive successively smaller probabilities p_2, p_3, p_4.

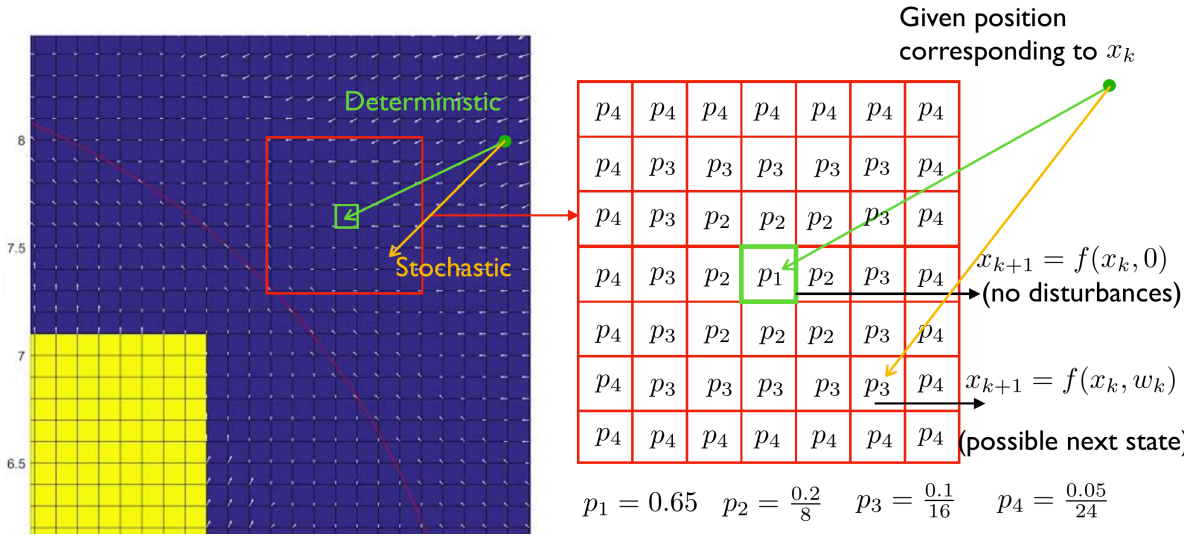


Fig. 11: Disturbance model: given the deterministic next cell (green), the actual next cell is uncertain and spreads over a neighbourhood with decreasing probability $p_1 > p_2 > p_3 > p_4$ as illustrated on the (zoomed-in) grid on the right.

Two different sensor models are considered for this example. In the first, the measurement $y_k = h(x_k, n_k)$ is itself uncertain and spreads over a neighbourhood of the true state x_k, in the same decaying-with-distance fashion as the disturbance model, with its own probabilities $q_1 = 0.5, q_2, q_3, q_4$ (Fig. 12). In the second, the measurement $y_k = h(x_k)$ is deterministic and simply reports whether there is an object or wall within a distance $M = 5$ cell-lengths of the robot, and if so, whether the closest one is to the left (L), right (R), up (U) or down (D) – or "no object close" (NO) if none is within range (Fig. 13).

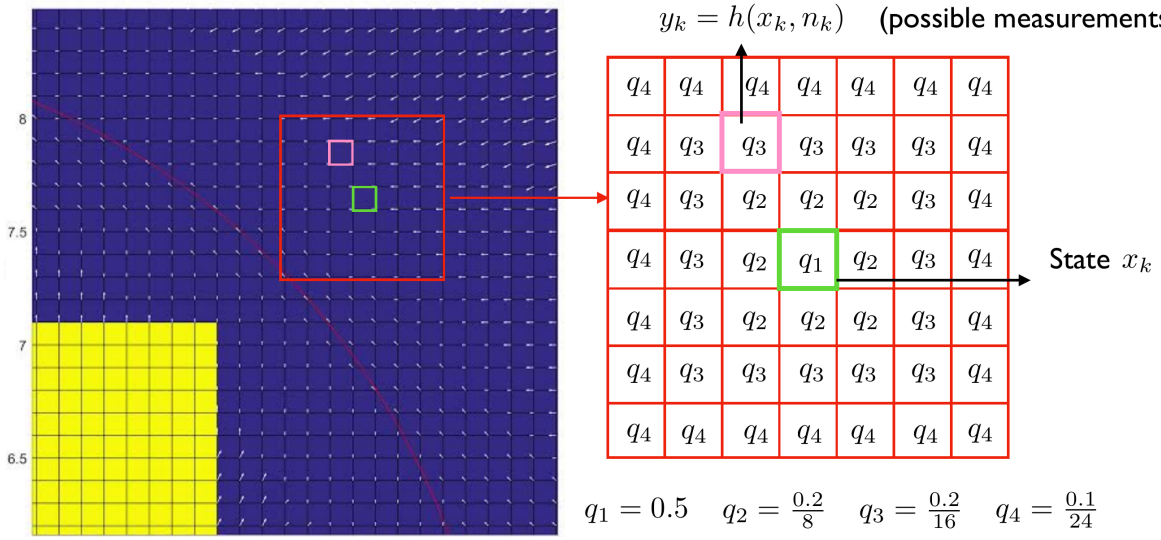


Fig. 12: Sensor model I: the measurement y_k spreads probabilistically over a neighbourhood of the true state, in the same fashion as the disturbance model (zoomed-in view on the right).

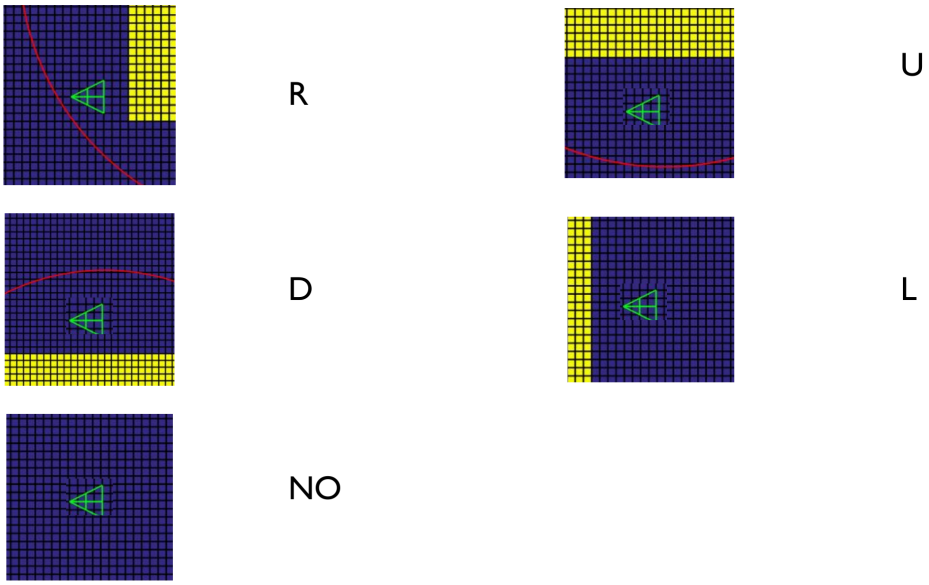


Fig. 13: Sensor model II: a deterministic measurement reporting the direction (Left, Right, Up, Down) of the nearest object/wall within a fixed range, or "No object close" (NO) if none is in range.

Running the Bayes filter on these two sensor models produces the videos below, which track the evolving belief over the grid as the robot moves and measurements arrive. In both, the left panel shows the environment – walls and the central obstacle in yellow – with the robot (green marker) driving anticlockwise along the red circular path, and prints the true position, the measurement y_k actually received, and the stage counter k. The right panel shows the belief $P_{x_k|I_k}$ that the filter maintains over the whole grid, brighter meaning more probable.

With **sensor model I**, the measurement is a noisy position: y_k is a pair of coordinates scattered around the true one, and the belief stays collapsed onto a near-point blob that follows the robot around the circle.

With **sensor model II**, the measurement is the deterministic directional reading, so y_k is a single symbol rather than a pair of coordinates and carries much less information per stage. The belief is correspondingly broader, and – more tellingly – it is *elongated*: at any moment it smears out along the direction that the current reading cannot resolve, stretching horizontally, diagonally or vertically as the robot works its way around the obstacle. Compare the two right-hand panels side by side; this is the picture to keep in mind of what a weaker sensor costs you.

! Interactive example: build your own environment and run the Bayes filter

The same ideas can be explored interactively. In the demo below the environment is yours to design: **click any cell** to turn it into a wall or clear it again, exactly as in the Dijkstra demo of Unit 3. The robot then moves stochastically through whatever environment you have built – at every stage it stays put or steps to one of its free orthogonal neighbours, all equally likely, which is the motion model of `A_4_K_gridworldBayes.ipynb` – and the Bayes filter tracks it using **sensor model II**: the robot only gets feedback when it is close to something, reporting the direction (U, D, L or R) of the nearest wall within M cells, or `NO` when nothing is within range.
The left panel shows the environment and the robot's true position (green), with a red arrow towards the wall it is currently sensing. The right panel shows the belief $P_{x_k|I_k}$, brightest where the robot is most likely to be, with the true cell outlined in red.
Because this sensor is *deterministic*, the correction step does something very easy to see: it simply rules out every cell whose reading differs from the one just received, and renormalizes over the rest. So

the bright region is exactly the set of positions still consistent with the whole history of readings, and the counter below the panels tracks how many those are. Watch what happens as the robot moves along a wall and then out into the open – in open space every cell reports `NO`, the measurements stop discriminating, and the belief spreads back out.

GPS-denied navigation in a tunnel

A second application is estimating the position of a car travelling through a GPS-denied tunnel (Fig. 14): GPS gives an accurate absolute position outside the tunnel, but is unavailable inside it, where dead-reckoning from the wheel odometry alone would accumulate drift over time – exactly the kind of situation where a Bayes filter, fusing whatever measurements remain available with a motion model, is valuable.

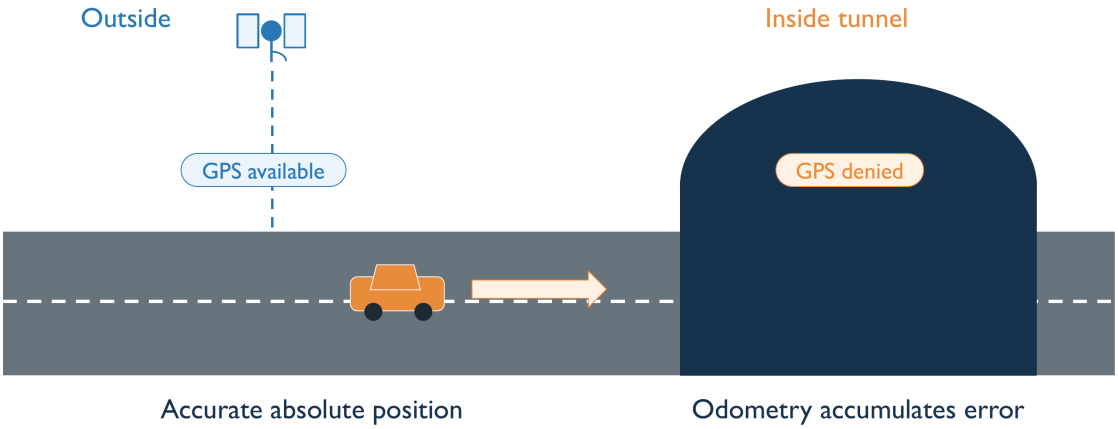


Fig. 14: Outside the tunnel, accurate GPS positioning is available; inside, GPS is denied and dead-reckoning odometry alone accumulates error over time.

In this model, position is discretized into cells along the lane, $q_k \in \{1, \dots, n\}$ (Fig. 15), and velocity into cell-jump units, $v_k \in \{0, 1, \dots, m\}$. An internal velocimeter provides a noisy velocity measurement $\tilde{v}_k = v_k - w_k$, with disturbance $w_k \in \{-n_w, \dots, -1, 0, 1, \dots, n_w\}$, so that the dynamics are $q_{k+1} = q_k + \tilde{v}_k + w_k$ (the true velocity v_k being the quantity actually driving the motion). Where available, GPS measurements are $y_k = q_k + n_k$, with noise $n_k \in \{-n_v, \dots, -1, 0, 1, \dots, n_v\}$. The Bayes filter for this model is set up exactly as in the previous examples, so the details are omitted here.



Fig. 15: Position discretized into cells $q_k \in \{1, \dots, n\}$ along the lane.

! Interactive example: watching the belief spread inside the tunnel

The demo below runs exactly this filter. The car drives left to right along the lane; the shaded span in the middle is the tunnel, where GPS is unavailable. The top strip shows the true cell q_k (red) and, when GPS is available, the reading y_k (orange tick above the lane). The bar chart underneath is the belief $P_{q_k|I_k}$ over the cells, with the true position marked by the red line.

Both noise sources are discretized Gaussians, $\text{Prob}[d] \propto e^{-d^2/2\sigma^2}$ truncated at $\pm 3\sigma$ and renormalized, and the two standard deviations are yours to set: σ_w for the velocimeter disturbance w_k, and σ_n for the

GPS noise n_k (the measurement noise, written v_k in some texts). Changing either starts a fresh run. The two have quite different jobs, and the demo is worth running a few times to see it. Inside the tunnel there is no correction step at all, so the belief is only ever propagated through the motion model and its spread grows steadily – this is dead-reckoning drift, and σ_w sets how fast it accumulates. The moment the car leaves the tunnel a single GPS reading collapses the belief again, and σ_n sets how sharply. With σ_w large and σ_n small you get the most dramatic version: a belief that fans out across a dozen cells in the tunnel and snaps back to near-certainty in one step on the way out.

Estimating plant attributes in precision farming

The Bayes filter is often used more broadly to keep track of an entire *world model*, not just a single robot's position. An example is the MSc thesis *Semantic SLAM for Autonomous Robots in Precision Farming Using Bayesian Inference* (Sander Doodman, TU/e, 2024), where it is used to keep track of the attributes of individual plants in an agricultural field, based on camera measurements gathered by a robot dog walking through the field (Fig. 16).



Fig. 16: A robot dog equipped with a RealSense L515 camera (left) used to gather measurements of individual plants for semantic mapping in precision farming (right); image from Sander Doodman's 2024 MSc thesis.

Solving POMDPs

Optimal policy structure

Returning to the general POMDP formulation, a general result (proved in the callout below) is that the optimal cost-to-go and policy can always be written in terms of a finite collection of n-dimensional vectors $\alpha_k^1, \alpha_k^2, \dots, \alpha_k^{a_k}$, as

$$J_k(p_k) = \min\{\alpha_k^{1\top} p_k,\ \alpha_k^{2\top} p_k,\ \dots,\ \alpha_k^{a_k\top} p_k\}, \qquad J_h(p_h) = \alpha_h^{1\top} p_h,$$

$$\mu_k(p_k) = \arg\min_{u_k}\ \mathbb{E}[g_k(x_k, u_k) + J_{k+1}(p_{k+1})].$$

That is, the optimal cost-to-go, as a function of the belief p_k, is always a *piecewise affine, concave* function – the pointwise minimum of finitely many linear functions of p_k – and the optimal policy partitions the belief simplex into regions, one per linear piece, within each of which a single decision is optimal.

i Justification: the cost-to-go is piecewise affine in the belief

For simplicity, assume the running cost $g(x_k, u_k)$ does not depend on time or on the disturbance, and that state, input and output live in fixed sets $x_k \in \{1, \dots, n\}$, $u_k \in \{1, \dots, m\}$, $y_k \in \{1, \dots, q\}$ over time.

The claim is proved by backward induction on k.
Base case. At the terminal stage, defining $\alpha_h^1 = \begin{bmatrix} g_h(1) & g_h(2) & \dots & g_h(n) \end{bmatrix}^\top$,

$$J_h(p_h) = \mathbb{E}[g_h(x_h) \mid I_h] = \sum_{i=1}^{n} g_h(i) \underbrace{\text{Prob}[x_h = i \mid I_h]}_{p_{h,i}} = \alpha_h^{1\top} p_h,$$

a single affine (indeed linear) function of p_h, as claimed.
Inductive step. Suppose the claim holds at stage $k+1$. Writing $\bar{\alpha}(u_k) = \begin{bmatrix} g(1,u_k) & \dots & g(n,u_k) \end{bmatrix}^\top$,

$$J_k(p_k) = \min_{u_k} \mathbb{E}[g(x_k,u_k) + J_{k+1}(p_{k+1}) \mid I_k] = \min_{u_k} \bar{\alpha}(u_k)^\top p_k + \sum_{\ell=1}^{q} \mathbb{E}[J_{k+1}(p_{k+1}) \mid I_k, y_{k+1} = \ell] \,\text{Prob}[y_{k+1} = \ell \mid I_k],$$

conditioning on the next measurement y_{k+1}. From the Bayes filter, $p_{k+1} = q_{k+1}/\text{Prob}[y_{k+1} = \ell \mid I_k]$ with $q_{k+1} = D(\ell) P_k(u_k) p_k$, and by the inductive hypothesis $J_{k+1}(p_{k+1}) = \min\{\alpha_{k+1}^{1\top} p_{k+1}, \dots, \alpha_{k+1}^{a_{k+1}\top} p_{k+1}\}$. Substituting and using that the normalizing factor $\text{Prob}[y_{k+1} = \ell \mid I_k]$ cancels against the division inside p_{k+1},

$$J_k(p_k) = \min_{u_k \in \{1,\dots,m\}} \bar{\alpha}(u_k)^\top p_k + \sum_{\ell=1}^{q} \left\{ \alpha_{k+1}^{1\top} D(\ell) P_k(u_k) p_k, \ \dots, \ \alpha_{k+1}^{a_{k+1}\top} D(\ell) P_k(u_k) p_k \right\},$$

where the inner braces denote that each of the a_{k+1} choices can be selected independently for each ℓ, before finally minimizing over u_k. Every term inside is linear in p_k, and minima and sums of linear functions combine into a new, larger finite collection of linear functions of p_k; hence

$$J_k(p_k) = \min\{\alpha_k^{1\top} p_k, \ \alpha_k^{2\top} p_k, \ \dots, \ \alpha_k^{a_k\top} p_k\}$$

for some vectors $\alpha_k^1, \dots, \alpha_k^{a_k}$, completing the induction.

In practice, however, the number of linear pieces a_k typically grows exponentially with the number of remaining stages, so this exact solution is mostly of theoretical interest, and heuristics are used to approximate POMDPs in applications. Solving a POMDP exactly, when feasible, proceeds in three steps:

1. Offline, determine the vectors $\alpha_k^1, \dots, \alpha_k^{a_k}$ for every stage k from the recursion above. These define linear functions on the n-dimensional probability simplex; using the parameterization $p = (q, 1 - \mathbb{1}^\top q)$, dropping the redundant last coordinate, these can be visualized directly as affine functions of q, as in Fig. 17.
2. Online, based on the measurements gathered up to time k, compute the current belief p_k with the Bayes filter.
3. Online, take the decision $\mu_k(p_k) = \arg\min_{u_k} \mathbb{E}[g_k(x_k,u_k) + J_{k+1}(p_{k+1})]$, using $J_k(p_k) = \min\{\alpha_k^{1\top} p_k, \dots, \alpha_k^{a_k\top} p_k\}$ precomputed in step 1.

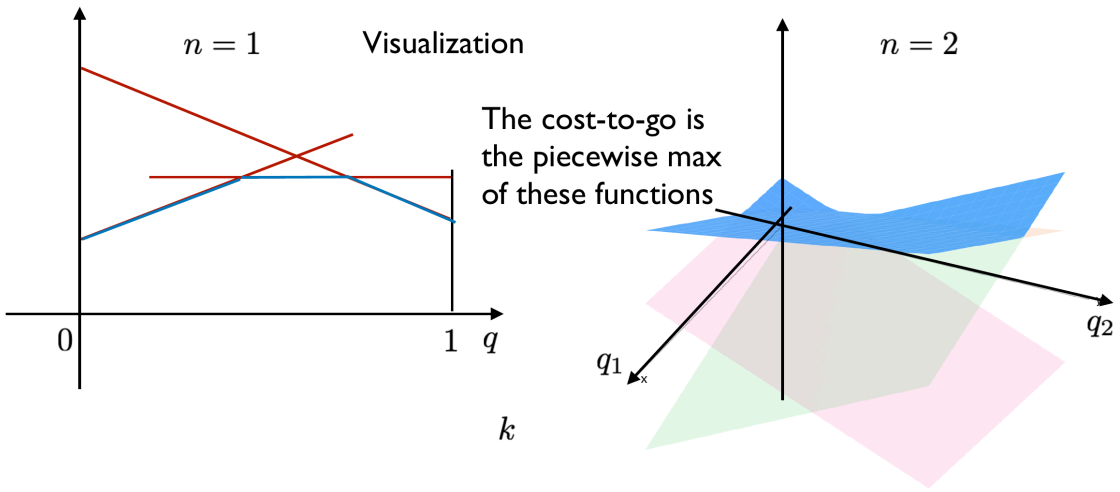


Fig. 17: The cost-to-go as the pointwise minimum of finitely many affine functions of the belief: one state-dimension left over after the simplex parameterization ($n = 1$, left) and two ($n = 2$, right).

Two worked examples of this procedure follow: a small, purely illustrative one (the Monty Hall problem) and a slightly larger one from Bertsekas' book (a machine-repair problem), both using the *first* approach discussed earlier in this unit – reformulating the problem with the information set as state – since for such small examples this is actually about as convenient as the belief-based approach, and arguably more transparent.

i Worked example: the Monty Hall problem

> Suppose you're on a game show, and you're given the choice of three doors: behind one door is a car; behind the others, goats. You pick a door, say No. 1, and the host, who knows what's behind the doors, opens another door, say No. 3, which has a goat. He then says to you, "Do you want to pick door No. 2?" Is it to your advantage to switch your choice?
> – Whitaker, Craig F. (9 September 1990, *Parade Magazine*: 16, via Wikipedia)

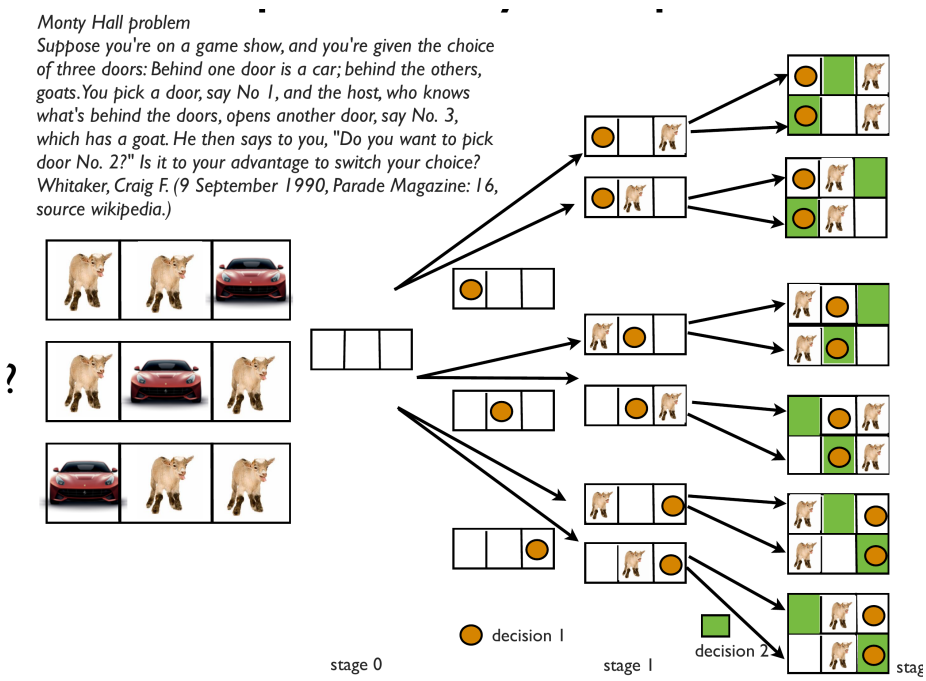


Fig. 18: The Monty Hall problem: three doors hide a car and two goats; after an initial pick, the host reveals a goat behind one of the other two doors.

Formulation. There are two decision stages, $k \in \{0, 1\}$ (equivalently $k \in \{0, 1, 2\}$ counting the terminal stage), with decisions $u_0, u_1 \in \{1, 2, 3\}$ (doors selected). The state has two "components": $\bar{x}_{k+1} = \bar{x}_k$, the door hiding the car, constant over time and unknown a priori (equally likely to be any of the three doors), and $\tilde{x}_{k+1} = u_k$, which simply keeps track of the previous decision. Formally, the overall state $x_k = (\bar{x}_k, \tilde{x}_k)$ takes one of 9 values, though $\tilde{x}_k$ and $\bar{x}_k$ can always be read off from x_k and vice versa. The cost is $g_0(x_0, u_0) = 0$, $g_1(x_1, u_1) = 1$ if $\bar{x}_1 = u_1$ and 0 otherwise (we want to *maximize* the probability of picking the car, so this cost is later negated), and $g_2(x_2) = 0$. There is no measurement at stage 0 ($y_0 = \emptyset$), and at stage 1 the host reveals a door $y_1 = h_1(x_1, n_1) \in \{1, 2, 3\}$ $\{\bar{x}_1, \tilde{x}_1\}$ if $\bar{x}_1 \neq \tilde{x}_1$ (the host must avoid both the car and the player's own pick), or a uniformly random door in $\{1, 2, 3\}$ $\{\bar{x}_1, n_1\}$ if $\bar{x}_1 = \tilde{x}_1$ (the player already picked the car, so the host picks freely between the two remaining doors, represented by the auxiliary random variable n_1), so that $I_1 = (y_1, u_0)$.

Computing the belief. To apply dynamic programming, start at stage 1 and compute $P_{\bar{x}_1|I_1}$ for every possible $I_1 = (y_1, u_0)$ (given I_1, $\tilde{x}_1 = u_0$ is immediate, so only $\bar{x}_1$ needs to be estimated). Take, for concreteness, $(y_1, u_0) = (3, 1)$: since the host never opens the door hiding the car, $\text{Prob}[\bar{x}_1 = 3 \mid y_1 = 3, u_0 = 1] = 0$ trivially. For the other two doors, Bayes' rule gives

$$\text{Prob}[\bar{x}_1 = 2 \mid y_1 = 3, u_0 = 1] = \alpha \cdot \text{Prob}[y_1 = 3 \mid \bar{x}_1 = 2, u_0 = 1] \cdot \tfrac{1}{3} = \alpha \cdot 1 \times \tfrac{1}{3} = \tfrac{2}{3},$$

$$\text{Prob}[\bar{x}_1 = 1 \mid y_1 = 3, u_0 = 1] = \alpha \cdot \text{Prob}[y_1 = 3 \mid \bar{x}_1 = 1, u_0 = 1] \cdot \tfrac{1}{3} = \alpha \cdot \tfrac{1}{2} \times \tfrac{1}{3} = \tfrac{1}{3},$$

with $\alpha = 1/\text{Prob}[y_1 = 3 \mid u_0 = 1] = 1/(\frac{1}{2} \times \frac{1}{2} + 1 \times \frac{1}{3}) = 2$ (Fig. 19). The same computation, repeated for every one of the six combinations of y_1 and u_0, always assigns probability $\frac{2}{3}$ to whichever door was *not* picked initially and was *not* opened by the host (Fig. 22 below).

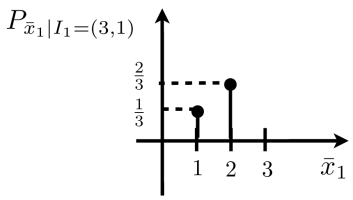


Fig. 19: The belief $P_{\bar{x}_1|I_1}$ after observing $(y_1, u_0) = (3, 1)$: probability $\frac{1}{3}$ that the car is behind door 1 (the player's original pick) and $\frac{2}{3}$ that it is behind door 2 (the only unopened, unpicked door).

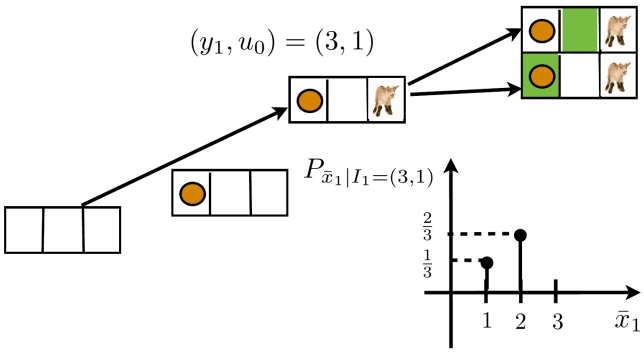


Fig. 20: The decision tree at stage 0: whichever door is initially picked, the host's reveal leads to two possible continuations, and the belief in each is the one computed above.

Optimal decision and policy. At stage 1, the optimal decision minimizes $\mathbb{E}[g_1(x_1, u_1)] = -\text{Prob}[\bar{x}_1 = u_1]$ (the negative sign turns the cost into a reward, since $g_1 = 1$ exactly when the guess is correct), which is minimized by picking the door with probability $\frac{2}{3}$ – i.e., by *switching* away from the original pick. Since this reasoning holds regardless of which door was picked initially and which was revealed, the optimal policy is simply: pick any door at stage 0, and always switch at stage 1 (Fig. 21). This wins the car with probability $\frac{2}{3}$, against only $\frac{1}{3}$ for never switching (Fig. 22).

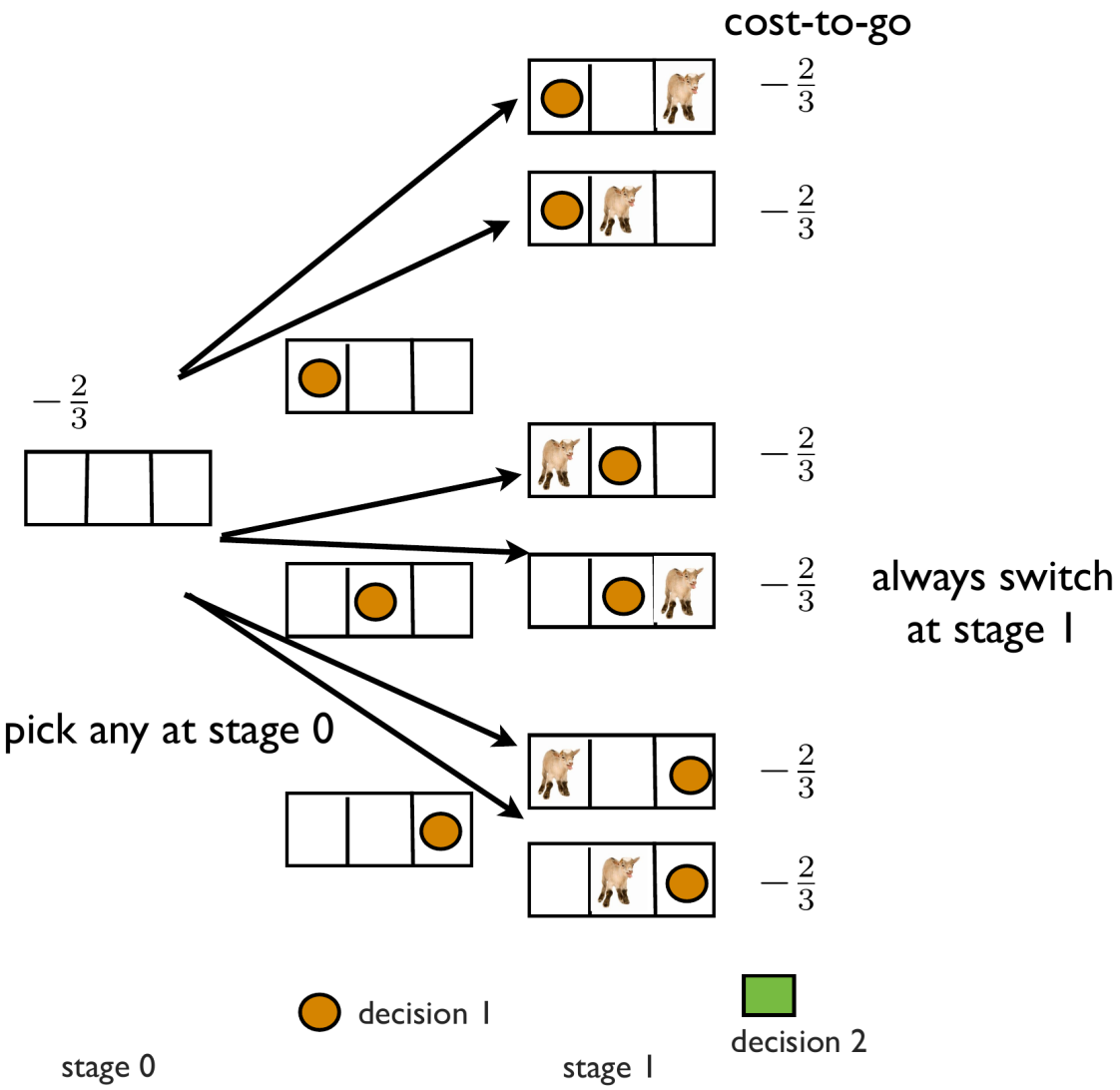


Fig. 21: The full policy tree: from every stage-0 pick and every possible reveal, switching (following the arrows to the un-opened, un-picked door) achieves cost-to-go $-\frac{2}{3}$.

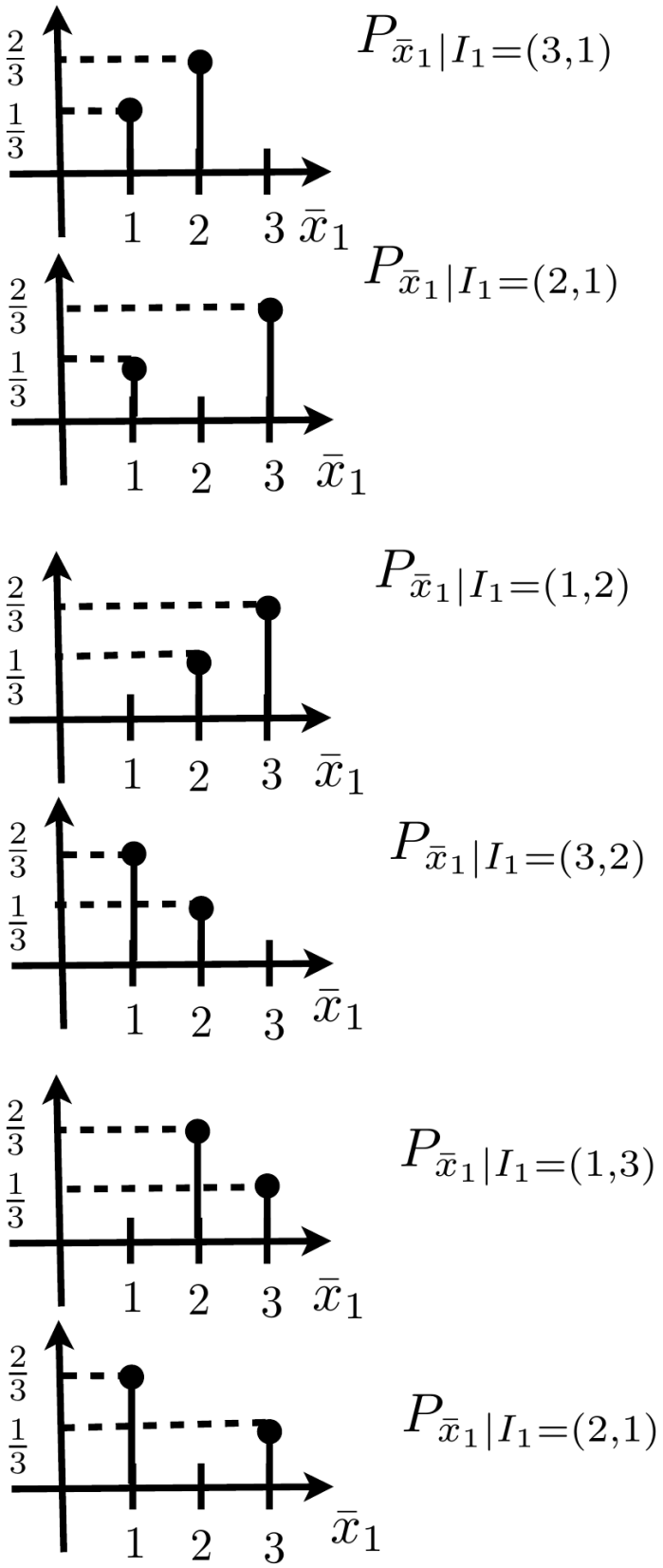


Fig. 22: The belief $P_{\bar{x}_1|I_1}$ for all six combinations of the initial pick u_0 and the host's reveal y_1: in every case, $\frac{2}{3}$ probability is on the unopened, unpicked door.

i Worked example: machine repair (Bertsekas, Section 5.1)

A machine can be in one of two states, P (proper condition) or $\bar{P}$ (improper condition) (Fig. 23). In each period, a machine starting in $\bar{P}$ stays in $\bar{P}$; one starting in P stays in P with probability $\frac{2}{3}$ and moves to $\bar{P}$ with probability $\frac{1}{3}$. The machine operates over 3 periods, starting in state P. At the end of the first and second periods there is an inspection, with outcome "good" G or "bad" B, $\text{Prob}[G \mid P] = \text{Prob}[B \mid \bar{P}] = \frac{3}{4}$, $\text{Prob}[B \mid P] = \text{Prob}[G \mid \bar{P}] = \frac{1}{4}$. After each inspection, the two possible actions are to continue operation, C (cost 0, state unchanged), or to stop and repair, S (cost 1, returns the state to P if it was $\bar{P}$); additionally, each period spent in $\bar{P}$ incurs a cost of 2, regardless of the action.

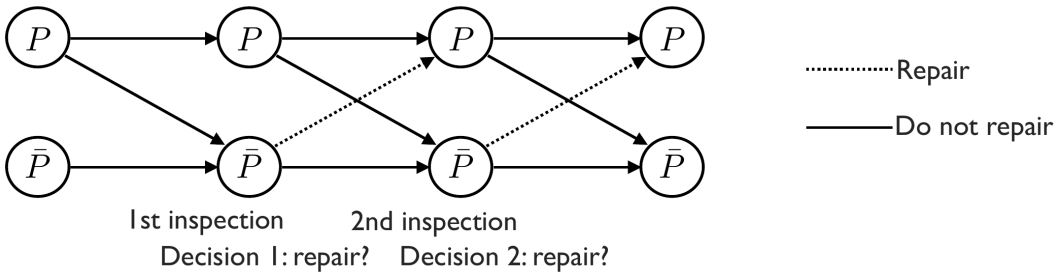


Fig. 23: The machine's state trajectory over two inspections, with the two possible actions – repair (dotted) or do not repair (solid) – shown at each decision point.

Formulation. With $x_{k+1} = w_k$, $k \in \{0, 1\}$, $x_k \in \{P, \bar{P}\}$, $u_k \in \{C, S\}$: $\text{Prob}[x_0 = P] = \frac{2}{3}$, $\text{Prob}[x_0 = \bar{P}] = \frac{1}{3}$; $\text{Prob}[w_k = P \mid x_k = P, u_k = C] = \frac{2}{3}$, $\text{Prob}[w_k = P \mid x_k = \bar{P}, u_k = C] = 0$; $\text{Prob}[w_k = P \mid x_k = P, u_k = S] = \text{Prob}[w_k = P \mid x_k = \bar{P}, u_k = S] = \frac{2}{3}$ (repair always resets the transition probabilities as if starting from P); measurements $y_k = v_k$ with $\text{Prob}[v_k = G \mid x_k = P] = \frac{3}{4}$, $\text{Prob}[v_k = G \mid x_k = \bar{P}] = \frac{1}{4}$; cost $g(P, C) = 0$, $g(P, S) = 1$, $g(\bar{P}, C) = 2$, $g(\bar{P}, S) = 1$; information sets $I_0 = (y_0)$, $I_1 = (y_0, y_1, u_0)$. The goal is to choose $\mu_0(I_0), \mu_1(I_1)$ minimizing $\mathbb{E}[g(x_0, \mu_0(I_0)) + g(x_1, \mu_1(I_1))]$.

Stage 1. Suppose the belief $p_1 = \text{Prob}[x_1 = \bar{P} \mid I_1]$ is known (so $\text{Prob}[x_1 = P \mid I_1] = 1 - p_1$). Action S has cost-to-go 1 regardless of p_1, since $g(P, S) = g(\bar{P}, S) = 1$; action C has expected cost $2p_1$ (only the $\bar{P}$ branch costs anything, weighted by its probability p_1). Hence

$$\mu_1(p_1) = \begin{cases} S, & p_1 > \frac{1}{2} \\ C, & p_1 \le \frac{1}{2} \end{cases} \qquad J_1(p_1) = \min(2p_1,\, 1),$$

already visibly piecewise affine in p_1, as the general theory predicts.

Stage 0. Suppose $p_0 = \text{Prob}[x_0 = \bar{P} \mid I_0]$ is known. The decision u_0 influences the cost-to-go only through how it shapes the distribution of p_1, once the next measurement y_1 is also taken into account: there are four combinations of $u_0 \in \{C, S\}$ and $y_1 \in \{G, B\}$, each producing a different $p_1 = \phi_1(p_0, u_0, y_1)$ via Bayes' rule (exactly as in the earlier robot examples):

$$p_1 = \phi_1(p_0, u_0, y_1) = \begin{cases} \frac{1}{7} & u_0 = S,\ y_1 = G \\ \frac{3}{5} & u_0 = S,\ y_1 = B \\ \dfrac{1 + 2p_0}{7 - 4p_0} & u_0 = C,\ y_1 = G \\ \dfrac{3 + 6p_0}{5 + 4p_0} & u_0 = C,\ y_1 = B \end{cases}$$

together with

$$\text{Prob}[y_1 = G \mid p_0, u_0 = C] = \frac{7 - 4p_0}{12}, \quad \text{Prob}[y_1 = B \mid p_0, u_0 = C] = \frac{5 + 4p_0}{12}, \quad \text{Prob}[y_1 = G \mid p_0, u_0 = S] = \frac{7}{12}, \quad \text{Prob}[y_1$$

The cost-to-go at stage 0 is then

$$J_0(p_0) = \min\Big[\, 2p_0 + \tfrac{7-4p_0}{12} J_1\big(\tfrac{1+2p_0}{7-4p_0}\big) + \tfrac{5+4p_0}{12} J_1\big(\tfrac{3+6p_0}{5+4p_0}\big), \quad 1 + \tfrac{7}{12} J_1\big(\tfrac{1}{7}\big) + \tfrac{5}{12} J_1\big(\tfrac{3}{5}\big) \,\Big],$$

the first branch for $u_0 = C$ and the second for $u_0 = S$, which simplifies (substituting $J_1(p_1) = \min(2p_1, 1)$ and working through the case boundaries) to

$$J_0(p_0) = \begin{cases} \frac{19}{12}, & \frac{3}{8} \le p_0 \le 1 \\ \frac{7}{12} + 4p_0, & 0 \le p_0 \le \frac{3}{8} \end{cases} \qquad \mu_0(p_0) = \begin{cases} C, & p_0 \le \frac{3}{8} \\ S, & p_0 > \frac{3}{8} \end{cases} \quad \text{(optimal policy)}.$$

Again J_0 is piecewise affine in p_0, with two linear pieces – consistent with the general theory, and illustrating concretely how quickly the number of pieces can grow with the number of remaining stages even in a tiny, two-stage example.

Historical note



Fig. 24: Thomas Bayes (1701-1761). (The only surviving portrait commonly said to be of Bayes, though its authenticity has been questioned; source: Wikipedia.)

Thomas Bayes was an English statistician, philosopher and Presbyterian minister, known for having formulated a specific case of the theorem that now bears his name. Bayes never published what would eventually become his most famous accomplishment; his notes on the subject were edited and published only after his death.

Summary

This unit introduced Partially Observable Markov Decision Processes, where the decision-maker only has access to a noisy, partial output of the state rather than the state itself, through an information set that grows with time. We saw that the optimal policy always decomposes into a state estimator, computing the belief $P_{x_k|I_k}$, and an actuator mapping this belief to a decision – and that the state estimator is exactly the Bayes filter, an alternating prediction/correction recursion built on the ordinary Bayes' rule. Most of the unit was devoted to this filter and its applications: tracking a robot from noisy measurements in a grid world, estimating position in a GPS-denied tunnel, and building a semantic world model in precision farming. We closed by showing that POMDPs can, in principle, always be solved exactly, since the optimal cost-to-go is a piecewise affine function of the belief – worked out concretely for the Monty Hall problem and a machine-repair example – though in practice the number of pieces grows exponentially with the number of stages, so this exact solution is mostly of theoretical interest and heuristics are used in applications.